An inference engine for eForth

version 1.0 - Thursday 22 October 2026



Marc PETREMANN

Table of Contents

Introduction **3**

The development environment **4**

The INFERENCE.FS file 4

The main.fs project 4

The other files 5

Defining facts **6**

Create a fact 6

Self-documenting a fact 7

Creation and management of the facts vocabulary 8

List of facts 8

Resetting the facts 9

Assigning a value to a fact 9

Define rules **11**

Create a rule 11

Self-documentation of rules 13

Creation and management of the rules vocabulary 14

Resetting the rules 14

Introduction

An **inference engine** is the central component of an expert system or symbolic artificial intelligence, responsible for automated decision-making and reasoning. It works by combining a **knowledge base** (made up of facts and production rules) with deductive algorithms to derive new information.

He primarily uses two reasoning strategies:

- **The chaining before** , which starts from the available facts to arrive at a conclusion.
- **The back-chaining** , which starts from a target hypothesis to work back to the necessary facts that validate it.

In short, it simulates human logic to solve complex problems without requiring rigid procedural programming.

The development environment

For the inference engine, we will use the eForth Windows version. It's a FORTH version usable for Windows, available here:

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__program

The documentation for this version is available here:

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__documentation

eForth requires no installation.

Simply download the **uEf64-7.0.7.21.exe file** into a directory, in the location of your choice, named for example **eForth** .

eForth directory . Create a new folder named **INFERENCE** . Download the necessary files from the software repository. You should have:

```
📁 eForth
    uEf64-7.0.7.21.exe
    INFERENCE.fs
📁 INFERENCE
    config.fs
    facts.fs
    inference-engine.fs
    main.fs
    rules.fs
    tests.fs
```

The INFERENCE.FS file

This is the file that allows you to connect to the rest of the project. Contents:

```
\ load INFERENCE
s" INFERENCE/main.fs" included
```

Each time you want to compile the code and use the inference engine, simply launch **eForth** by clicking on the **uEf64-7.0.7.21.exe file** . Once eForth has started, type:

```
include INFERENCE.fs
```

The main.fs project

The entire project is compiled from **main.fs.** Example content:

```
include config.fs            \ load config
include inference-engine.fs  \ load inference engine
```

You can add the loading of other files. For example, to compile **tests.fs** :

```
include config.fs            \ load config
include inference-engine.fs  \ load inference engine
include tests.fs
```

The other files

They are loaded from **main.fs.** The project is divided into functionalities that can be used as the project evolves. The list and content of the source files may change without notice and may no longer be those described in this document.

facts.fs cannot be loaded before the contents of **inference-engine.fs** have been processed .

Defining facts

Think you've mastered logic? That's perfect. So you know how to manipulate the logical operators **AND** , **OR** , **NOT** … Let's look at a practical case with a completely trivial example:

- the temperature is very cold
- it's raining

We have two facts. We can deduce that if it rains and it is very cold, the risk of frost is real.

This can be coded in FORTH:

```
 0 constant FALSE
-1 constant TRUE

VARIABLE rain
VARIABLE cold-weather
VARIABLE freeze-

: RESET-FACTS ( -- )
    FALSE rain !
    FALSE cold-weather !
    FALSE fact-freezing !
```

Our variables, rain **cold-weather** and **fact-freezing**, are the flags to be tested, with values of 0 for false and -1 for true. We can apply these rules to these two facts:

```
\ Defining the rules as standard words
: cond-rule1 ( -- flag )
    rain @  cold-weather @  AND ;
: act-rule1  ( fl -- )
    cond-regle1 if
        TRUE fait-gel !
    then ;
```

At this stage, we cannot yet speak of inference. And a single rule, here **act-rule1**, does not justify the creation of an inference engine.

Create a fact

This is the first step towards an inference engine. A fact is a variable that contains a logical state or a numerical value. To define a fact:

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value
  ;
```

This allows us to rewrite the variables as follows:

```
FALSE FACT: rain
FALSE FACT: cold-weather
FALSE FACT: freeze
```

The word **FACT:** creates a fact, which is simply a value. Executing a fact adds its value to the stack.

```
21 FACT: temperature
temperature   \ push on stack 21
```

Self-documenting a fact

The idea is to append a string of characters that provides information about the nature of the content of the declared fact. EForth can handle zero-terminated strings:

```
z" Outdoor Temperature" z>s type  \ displays text: Outdoor Temperature
```

To attach this type of string to a fact, we simply compile the address of this string after the value:

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value                   \ create value
    ,                       \ compile pointer to zstring
  ;
```

And it's used like this:

```
z" Outdoor Temperature" 21 FACT: ext-temp
```

The execution of **ext-temp** pushes the value 21 onto the stack.

To retrieve the compiled string following this value, you need to explore the data field of **ext-temp**:

```
' ext-temp >body cell + @ z>s cr type
\ display: Outside temperature
```

To facilitate access to data in the data field of a fact created by **FACT:**, we define this structure:

```
structures
struct __FACTS
    ptr field ->value       \ fact value
    ptr field ->factZstring     \ point to fact zstring
forth
```

To determine the status of a fact, we will define this word:

```
\ display fact zstring and value in fact
\ example: ' <fact> explain-fact
: explain-fact { xtAddr -- }
    xtAddr >name type
    xtAddr >body ->factZstring @ z>s cr  4 spaces type ."  = "
    xtAddr >body ->value @ { value }
    value 0= if
        ." FALSE"
    then
    value -1 = if
        ." TRUE"
    then
    value 0=  value -1 = or invert if
        value .
    then
```

```
    cr cr
  ;
```

And it is used like this:

```
' ext-temp explain-fact  \ display:
\ ext-temp
\     Température = 21
```

Creation and management of the facts vocabulary

Currently, all facts are defined in the **FORTH** vocabulary . If we mix classic definitions, rules, and facts, running **vlist** will produce a list of words that is difficult to manage. The idea would therefore be to define all facts, and only those facts, in a specific vocabulary:

```
vocabulary facts    \ new facts defined in this vocabulary
```

To switch a fact creation to this vocabulary, vocabulary selectors are defined:

```
: in-facts ( -- )
    facts definitions
  ;

: in-forth ( -- )
    only
    forth definitions
  ;
```

These selectors are incorporated into the definition of **FACTS:**

```
\ create a new fact
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    in-facts
    value                   \ create value
    ,                       \ compile pointer to zstring
    in-forth
  ;
```

Let's define these facts:

```
z" Rain it" FALSE   FACT: rain
z" Outside temperature" -4   FACT: cold-weather
z" Is freezing" FALSE  FACT: freeze
```

These facts will be defined in the **facts** vocabulary and are visible:

```
facts vlist \ display: freeze cold-weather rain
```

List of facts

The underlying idea of fact management in the vocabulary of **facts** is to enable detailed listing. To do this, we define this word:

```
forth definitions also internals
: list-facts ( -- )
    facts context @ @
    begin  dup nonvoc?
```

```
    while  dup explain-fact cr cr >link
    repeat
    drop
    forth
  ;
only
```

The execution of **list-facts** selects the vocabulary **facts** and retrieves the executable code for each fact. This executable code is then used by **explain-fact**. Result:

```
list-facts    \ display:
\ freeze
\     Is freezing = FALSE
\
\  Outside temperature
\     cold-weather = -4
\
\ Rain it
\     rain = FALSE
```

This list of facts displays the facts, and on the following line, the self-documentation and the value of the fact. The keyword **list-facts** thus allows you to trace all the facts processed by the inference engine.

Resetting the facts

The way the facts are listed is exploited to return all the facts to **FALSE** :

```
\ reset fact value to FALSE
: (reset-fact) ( xt -- )
    >body FALSE swap !
  ;

\ reset values of all facts defined in facts vocabulary
forth definitions also internals
: reset-facts ( -- )
    facts context @ @
    begin  dup nonvoc?
    while  dup (reset-fact) >link
    repeat
    drop cr ." All facts set to FALSE"
    forth
  ;
only
```

The word **reset-facts** analyzes each fact declared in the **facts** vocabulary and resets its value to **FALSE**.

Assigning a value to a fact

The word **val-input** is defined in the **vocForthExtensions.fs** file :

```
\ wait a value and push it on stack
: val-input ( -- n )
    PAD 80 ACCEPT
    PAD SWAP
    EVALUATE
```

```
  ;
```

vocForthExtensions.fs file are compiled by **main.fs** before the inference engine is compiled. The **`val-input`** word waits for a numeric value to be entered from the keyboard and pushes its value onto the stack. Usage:

```
facts
cr." Current temperature: " val-input to ext-temp \ enter 27
ext-temp .   \ displays 27
```

Note that you can also enter **`TRUE`** or **`FALSE`** , because **`val-input`** evaluates the entered value. If this value is a constant or a value from the FORTH vocabulary, that word will be executed and its value placed on the data stack.

Define rules

The rules address the facts in two stages:

- verifies one or more facts;
- deals with one or more facts if the rule is verified

Example :

```
FALSE value F1
FALSE value F2
FALSE value F3
: R1  ( -- )   \ our first rule
    F1 F2 and if
        TRUE to F3
    then ;
```

Nothing too complicated. Let's factor out the *condition* part and the *action* part :

```
: condition-R1 ( -- fl )
    F1 F2 and ;
: action-R1 ( -- )
    TRUE to F3 ;
: R1  ( -- )   \ our first rule
    condition-R1 if
        action-R1
    then ;
```

Create a rule

The idea is to define a word **`(RULE:)`** that manages a condition and an action. Code for this word:

```
structures
struct __RULES
    ptr field ->condition   \ Rule condition field
    ptr field ->action      \ field of the rule's action part
    ptr field ->execFlag    \ rule execution flag field, set to TRUE if rule
executed
forth

\ Executes the condition
: exec-condition ( addr -- fl )
    ->condition @ execute
  ;

\ execute the action
: exec-action ( addr -- )
    ->action @ execute
  ;

\ Mark rule as triggered
\ Indicate that a rule triggered in this cycle
: set-flag { addr -- }
    TRUE addr ->execFlag !
    TRUE to MODIFIED_RULE
  ;\ create a new rule
```

```
: (RULE:)
    create
        swap , ,    \ store xt condition and xt action
        0 ,         \ store initial flag, value FALSE
    does>
        { addr }
        \ Execute rule if has not yet been triggered in the cycle
        addr ->execFlag @ 0= if
            addr ->condition @ execute if
                addr ->action @ execute
                addr set-flag
            then
        then
  ;
```

In the word **(RULE:)** , the part that follows **CREATE** :

- we store the execution code of the condition,
- We then store the execution code for the action part,
- Finally, we store a flag which is switched to **TRUE** if the rule is invoked.

Example of a rule definition:

```
:NONAME loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:NONAME loanAmount loanTerms interestRate MENSUALITE-EXACTE to
monthlyInstallment ;
RULE: calcMonthlyInstallment
```

The sequence **:NONAME <definition> ;** places the execution code of the FORTH code placed after **:NONAME** on the stack .

To make the creation of a rule more readable, we define these two words:

```
\ define condition of a rule
: :CONDITION ( -- fl )
    facts
    :NONAME
  ;

\ define action for a rule
: :ACTION ( -- )
    facts
    :NONAME
  ;
```

This allows us to rewrite the definition of our rule:

```
:CONDITION loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:ACTION    loanAmount loanTerms interestRate MENSUALITE-EXACTE to
monthlyInstallment ;
(RULE:) calcMonthlyInstallment
```

Our **calcMonthlyInstallment** rule executes sequentially:

- analyzes the execution flag, which is initially zero. If the flag is null, it means the rule has not yet been processed;

- In this case, execute the condition code. This code must always issue a **TRUE** or **FALSE** flag .
- If the condition flag is set to **TRUE** , execute the action code.

In our example, we test three facts: **loanAmount** , **loanTerms**, and **interestRate** . If these facts are not equal to zero, the condition is true. In this case, the action is executed. Here, we calculate the monthly loan payment and assign the result to the **monthlyInstallment** fact. Then, the rule's execution flag is set to **TRUE**. When this flag is true, the rule can no longer be executed.

Another **MODIFIED_RULE** flag is set to true. This is a global flag. It is set to **FALSE** for each inference. If only one rule is executed, it is set to **TRUE** . We will see later how this flag is managed by the inference engine.

Self-documentation of rules

As with facts, it is easy to self-document a rule:

```
z" Calculated monthly loan payment"
:CONDITION loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:ACTION    loanAmount loanTerms interestRate EXACT-MONTHLY-PAYMENT to
monthlyInstallment ;
RULE: calcMonthlyInstallment
```

Self-documentation of a rule is integrated after **CREATE** in the rule definition, as shown in the code excerpt:

```
\ create a new rule
: (RULE:)
    create
        swap , ,    \ store xt condition and xt action
        0 ,         \ store initial flag, value FALSE
        ,           \ store auto-documentation string pointer
    does>
        \ DOES code here
  ;
```

To display this self-documentation, we will use the word explain-rule, preceded by the execution code of the rule to be displayed:

```
\ display rule zstring and name in rules
\ example: ' <rule> explain-rule
: explain-rule { xtAddr -- }
    xtAddr >name type
    xtAddr >body ->ruleZstring @ z>s cr  4 spaces type ."  = "
    xtAddr >body ->execFlag @
    0= if
        09 bg ."  KO " 0 bg     \ red background
    else
        10 bg ."  OK " 0 bg     \ green background
    then
    cr cr
  ;
```

Example :

```
' calcMonthlyInstallment explain-rule
```

Display :

```
calcMonthlyInstallation
Calculated monthly loan payment = KO
```

Here, the KO result indicates that the rule has not yet been executed.

Creation and management of the rules vocabulary

Just as we defined the vocabulary **facts** , we define and use the vocabulary **rules**:

```
vocabulary rules    \ new rules defined in this vocabulary
```

To switch rule creation to this vocabulary, you define your vocabulary selector:

```
: in-rules ( -- )
    rules definitions
  ;
```

This **in-rules** selector is used as follows:

```
\ define new rule in rules vocabulary
: RULE: ( -- )
    in-rules
    (RULE:)
    in-forth
  ;
```

Let's return to our example of rule creation:

```
z" Calculated monthly loan payment"
:CONDITION loanAmount 0 >  loanTerms 0 > and interestRate 0 > and ;
:ACTION    loanAmount loanTerms interestRate EXACT-MONTHLY-PAYMENT to
monthlyInstallment ;
RULE: calcMonthlyInstallment
```

Now our **calcMonthlyInstallment** rule is defined in the **rules** vocabulary.

To retrieve all the rules defined in the `rules vocabulary` , we will execute the following:

```
rules vlist   \ display:
hightDebtTest evalHightIncome calcMonthlyInstallment getLoanAmount getMonthIncome
```

At this stage, we could also define **list-rules** in a similar way to **list-facts**. We will see a significant improvement in their definition in the next chapter.

Resetting the rules

Here is the code that resets all rule execution flags to **FALSE** :

```
\ Resets the execution flag for all rules
forth definitions also internals
: reset-rules ( -- )
    rules context @ @
    begin dup nonvoc?
```

```
    while
        FALSE over >body ->execFlag !  \ set ->execFlag à FALSE
        >link
    repeat
    drop
  ;
only
```

The word **reset-rules** analyzes each rule declared in the **rules** vocabulary and resets its value to **FALSE**.